BranchBox: Stage and Target Supervision for Multi-Object Placement Supplementary Material

Qinzhen Ma[1*] Harry Yang[2*] Changsheng Lu[2*] Xiaogang Xu[3]
[1]Rice University [2]The Hong Kong University of Science and Technology
[3]Zhejiang University
[*]Equal contribution.

Research preprint, November 2026.

1. All configurations

The main paper gives an exploratory analysis of SmolVLA and X-VLA on four two-object LIBERO placement tasks, selected after the broader study. The original primary analysis below did not meet its prespecified success criterion. We ran the same recipe on three models and ten tasks, 1,740 episodes in all. Table 1 gives every cell. Evo-1 LIBERO P0 and PS ran the first 15 initial states under a time rule fixed in advance, whose deadline moved by one hour after the host reboot, on timing grounds only; contrasts use the common states.

Task	SmolVLA N0/P0/PS	X-VLA N0/P0/PS	Evo-1 N0/P0/PS
Moka pots	14 / 17 / 15	11 / 16 / 14	0 / 0 / 0
Microwave	17 / 16 / 12	17 / 13 / 14	0 / 0 / 0
Bottom drawer	18 / 18 / 17	18 / 18 / 20	0 / 1 / 0
Two mugs	8 / 9 / 10	7 / 7 / 11	0 / 0 / 0
Mug and pudding	10 / 13 / 13	8 / 9 / 8	0 / 0 / 0
Soup and sauce	10 / 12 / 12	5 / 6 / 8	0 / 0 / 0
PreSoakPan (RoboCasa365)	1 / 1 / 0	1 / 0 / 0	0 / 0 / 0
LoadDishwasher (RoboCasa365)	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0
ArrangeTea (RoboCasa365)	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0
PanTransfer (RoboCasa365)	1 / 0 / 0	0 / 0 / 0	0 / 0 / 0

Table 1. Successes per cell, out of 20 episodes, except Evo-1 LIBERO P0 and PS, out of 15.

Contrast (points)	All ten tasks	LIBERO	RoboCasa
PS−N0	+1.3 [−1.5, +4.2]	+3.1 [−1.7, +7.8]	−1.3 [−2.9, 0.0]
PS−P0	−0.4 [−3.1, +2.3]	−0.4 [−4.8, +4.1]	−0.4 [−1.3, 0.0]
P0−N0	+1.7 [−1.1, +4.6]	+3.4 [−1.3, +8.1]	−0.8 [−2.5, +0.8]
	SmolVLA	X-VLA	Evo-1
PS−N0	0.0 [−5.5, +5.5]	+4.0 [−2.0, +10.5]	0.0 [0.0, 0.0]
PS−P0	−3.5 [−9.0, +2.0]	+3.0 [−2.5, +8.5]	−0.7 [−2.0, 0.0]
P0−N0	+3.5 [−2.5, +9.5]	+1.0 [−5.0, +6.5]	+0.7 [0.0, +2.0]

Table 2. Contrasts over all three models, equal weight, stratified by model×task. Our criterion set in advance, a lower bound above zero with at least two models positive for PS−N0 over all ten tasks, is not met.

	N0 native	P0 trained	PS self-prompt	PS−N0 (points)	P0−N0 (points)	PS−P0 (points)
SmolVLA	77/120 (64.2%)	85/120 (70.8%)	79/120 (65.8%)	+1.7 [−7.5, +10.8]	+6.7 [−3.3, +16.7]	−5.0 [−14.2, +4.2]
X-VLA	66/120 (55.0%)	69/120 (57.5%)	75/120 (62.5%)	+7.5 [−2.5, +17.5]	+2.5 [−7.5, +12.5]	+5.0 [−4.2, +14.2]
Both models	143/240 (59.6%)	154/240 (64.2%)	154/240 (64.2%)	+4.6 [−2.5, +11.7]	+4.6 [−2.5, +11.7]	0.0 [−6.7, +6.3]

Table 3. Task success on six long-horizon LIBERO tasks. Brackets are 95% paired bootstrap intervals; differences are in percentage points. This two-model scope was chosen after the runs; the comparison is exploratory. Results for all configurations we ran are in the supplementary material.

Task	N0	P0	PS	PS−N0
Moka pots	25/40	33/40	29/40	+10.0 [−7.5, +27.5]
Microwave	34/40	29/40	26/40	−20.0 [−35.0, −5.0]
Bottom drawer	36/40	36/40	37/40	+2.5 [−7.5, +12.5]
Two mugs	15/40	16/40	21/40	+15.0 [−2.5, +32.5]
Mug and pudding	18/40	22/40	21/40	+7.5 [−12.5, +27.5]
Soup and sauce	15/40	18/40	20/40	+12.5 [−5.0, +30.0]

Table 4. Success per task, summed over SmolVLA and X-VLA; differences in percentage points, with 95% paired bootstrap intervals in the last column.

2. Complete six-task LIBERO comparisons

Tables 3 and 4 retain the complete six-task results for the two models emphasized in the main paper. These tables and their intervals are unchanged from the earlier broader analysis. In particular, P0 and PS each solve 154/240 episodes over all six tasks; the feedback-only contrast is 0.0 points with 95% CI $[-6.7, +6.3]$. The four-task main analysis must therefore be interpreted within its stated task scope, not as evidence of a uniform benchmark-wide effect or an isolated benefit of feedback.

3. Focused-analysis reproducibility

The four focused tasks are the two-object placement tasks in the evaluated six-task LIBERO set. Their internal IDs are 0, 3, 4 and 5, corresponding to LIBERO-Long IDs 8, 4, 6 and 0. This semantic grouping and the two-model focus were chosen after results were available. Every focused task–initial-state pair has the same initial-frame hash and episode seed across both models and all three arms. Thus the 160 observations per arm comprise two model evaluations of each of 80 shared scenes.

The focused analysis uses NumPy default_rng with seed 20261101 and $B = 100{,}000$. In task order 0, 3, 4, 5, we draw a $B \times 20$ matrix of uniform indices from 0–19 for that task; the same indices select outcomes for both models and all arms. Each replicate averages success equally across the eight model–task cells. Contrasts are differences of these paired averages; intervals are the 2.5th and 97.5th percentiles. The source package's `analysis/episodes.csv` contains the 720 binary outcomes for both models on all six LIBERO tasks, and `analysis/reproduce.py` reconstructs all focused counts and intervals from those rows. This focused reanalysis uses existing rollouts. The additional memory experiment in Sec. 10 uses a separate development panel.

As a sensitivity check, independent resampling within model–task cells gives PS−N0 $+11.25$ points with CI $[1.875, 20.0]$, versus $[1.875, 20.625]$ with common-scene resampling. For P0−N0, the corresponding intervals are $[1.25, 18.75]$ and $[0.0, 19.375]$. We use the common-scene intervals throughout the revised main paper and make no confirmatory significance claim. Historical broader-study tables retain their original analysis.

For the focused family, PS−P0 is $+1.25$ points with an unadjusted common-scene 95% CI of $[-8.1, +10.0]$ (28 discordant wins and 26 losses). The main comparisons against N0 concern the combined training recipe under fixed deployment choices. Relative improvement uses unrounded counts: $(91 - 73)/73 \approx 24.6575\%$, rounded to 24.7%; the absolute gain is $100(91 - 73)/160 = 11.25$ percentage points, rounded to 11.3. All focused intervals condition on the fixed checkpoints and tasks; they do not account for retrospective selection or training-seed variability.

4. Stage progress

Table 5 gives the share of episodes that reach the final stage. Reaching it shows how far the filter advanced, not that the task progressed.

Cell	P0	PS
SmolVLA LIBERO	92.5%	90.0%
SmolVLA RoboCasa	92.5%	85.0%
X-VLA LIBERO	72.5%	65.8%
X-VLA RoboCasa	70.0%	58.8%
Evo-1 LIBERO	30.0%	24.4%
Evo-1 RoboCasa	98.8%	33.8%

Table 5. Episodes reaching the final stage. On Evo-1 RoboCasa feedback was evaluated, but the stage filter often stalled; this is a failure of the tested loop, not a missing evaluation.

5. Exploratory subgroup selected by native success

The following grouping was defined after inspecting the same N0 outcomes used in the contrasts. Intervals do not account for that selection. The three low-native-success tasks are two mugs, mug and pudding, and soup and sauce; all are living-room tasks. Their apparent advantage can reflect scene effects and regression to the mean. Two of the ten even splits of the six tasks have a PS−N0 difference at least as large; this descriptive split comparison is not a selection-adjusted significance test.

Group	N0	P0	PS	PS−N0 (points)
Native $\leq 50\%$	48/120	56/120	62/120	$+11.7$ $[+0.8, +22.5]$
Native $> 50\%$	95/120	98/120	92/120	-2.5 $[-10.8, +5.8]$

Table 6. Post hoc subgroups, pooled across SmolVLA and X-VLA. Brackets are unadjusted 95% paired bootstrap intervals.

6. Training-data construction

The six LIBERO tasks have 300 original demonstrations in total. The data-generation pipeline removes no-op actions before replay and retains only demonstrations whose replay ends in success. The retained counts, in the order of the first six rows of Table 1, are 29, 34, 36, 37, 35 and 32, totaling 203 demonstrations and 58,272 frames. Thus 97 demonstrations are excluded by the replay-success filter. Both N and P train on the same retained set; this controls their local comparison but differs from training on the complete official demonstration set. Label quality and the effect of this filtering are not separately evaluated.

Training and inference details. The training seed is 1 for every configuration. LIBERO training budgets are 14,000, 20,000 and 10,000 updates for SmolVLA, X-VLA and Evo-1, respectively; RoboCasa budgets are 13,000, 13,000 and 6,500. All use batch size 32. Evo-1 uses a newly initialized action head with a frozen visual-language backbone, rather than a fully fine-tuned action-policy checkpoint. RoboCasa uses 2,000 demonstrations and 1,449,081 frames; the stated budgets cover less than one pass through those frames. The time-limited design is therefore not a comparison at converged performance.

7. Implementation details

Supplied task tables. Table 7 lists the exact subtask strings for the four focused tasks. The task table is constructed from the existing demonstration annotation metadata before policy training. Within each row, stages 0 and 1 share the first string, and stages 2 and 3 share the second string. The mark sequence is crosshair, box, crosshair, box. A crosshair refers to the moved object and a box to its destination. The table supplies both the semantic decomposition and its order at test time.

Task (ID)	Stage-0/1 text	Stage-2/3 text
Moka pots (8)	put the second moka pot on the stove	put the first moka pot on the stove
Two mugs (4)	put the white mug on the left plate	put the yellow and white mug on the right plate
Mug/pudding (6)	put the white mug on the plate	put the chocolate pudding to the right of the plate
Soup/sauce (0)	put the alphabet soup in the basket	put the tomato sauce in the basket

Table 7. Exact supplied texts; IDs are zero-based task indices in `libero_10`. The prompt appends a newline followed by `Current subtask:`, the selected string, and a period to the full instruction.

Offline LIBERO annotations. The labeler follows the fixed per-task order and skips goal conjuncts already true at the demonstration's initial frame. A subtask advances at the first frame of a five-frame run in which its simulator predicate is true; a shorter true run that reaches the final state also qualifies. This rule uses future demonstration frames offline. Reach-to-place phase transition requires both-finger contact and a lift strictly greater than 1 cm relative to the object height at the start of that contact run. Contact gaps of at most two frames, bounded by contact on both sides, are filled. Within an unchanged subtask, the phase returns to reach after ten consecutive not-held frames; the raw phase is reevaluated on a subtask change. The resulting stage label is twice the subtask position plus the phase.

The reach target is the visible-mask centroid of the moved object. The place target is the destination entity's visible segmentation bounding box, with region-site clipping where applicable. For a tabletop region, the box is instead the image projection of the region site's eight corners, clipped to the image. Entity masks with fewer than four visible pixels have no target label. Mask centroids use pixel centers; box maxima are exclusive pixel edges. Coordinates are normalized in the stored main-camera frame. LIBERO images are rotated by 180 degrees from the raw simulator orientation before policy use; the prompt renderer does not rotate them again. Predicates, contacts and segmentation label demonstrations only and are not inputs to the test-time prompter.

Model adapters and training. SmolVLA starts from `HuggingFaceVLA/smolvla_libero`. Its three-view configuration adds the prompt camera while reusing the initial weights unchanged. X-VLA starts from `lerobot/xvla-base`. Table 8 gives the adapter settings. Both N and P use the same three camera slots and auxiliary-head architecture. At zero auxiliary weight, the auxiliary heads run on detached features without gradients. Dataset image augmentations are disabled to preserve the alignment of numeric targets and rendered marks; model-native image preprocessing remains in place. State and action statistics for the retained six-task dataset are recomputed before training.

Setting	SmolVLA	X-VLA
Trained policy modules	Action expert and state/action projections	Policy transformer and soft prompts
Frozen modules	Visual/language backbone	All of `model.vlm`, including multimodal projector; kept in evaluation mode
Auxiliary readout	Input to `model.action_out_proj`	Input to `model.transformer.action_decoder`
Feature reduction	Mean over chunk tokens	Mean over chunk tokens
Chunk / executed actions	50 / 10	30 / 30
LIBERO updates	14,000	20,000
Batch / training seed	32 / 1	32 / 1
Maximum stale-label lag	30 frames	90 frames
State normalization	Mean/std from training dataset	Identity; padded to 20 dimensions
Action interface	7-D LIBERO delta action	`action_mode=auto`; 20-D decoder with MSE on the 7 real action dimensions
Additional setting	Initial weights reused for the extra view	Domain ID 10; float32 weights with bf16 autocast

Table 8. Configuration details of the focused models. Auxiliary readouts are forward pre-hook inputs, not predicted actions or visual-encoder features. Each head has two 256-unit ReLU hidden layers.

Saved optimizer configurations. For each model, the saved N and P training configurations agree except for output directory and job name. SmolVLA uses AdamW and X-VLA uses the native `xvla-adamw` variant, both with peak learning rate 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$ and gradient clipping at 10. Weight decay is 10^{-10} for SmolVLA and 10^{-4} for X-VLA; the latter's soft-prompt learning-rate multiplier is one. Both schedules specify 1,000 warmup updates and cosine decay configured over 30,000 updates towards 2.5×10^{-6}. The actual budgets are 14,000 and 20,000, respectively, so the runs do not reach that configured endpoint. The original checkpoints do not record a pretrained-model or dataset revision commit; no revision hash is inferred from the repository names. These settings describe the original two-model experiment, distinct from the additional full-weight memory training.

LIBERO proprioception has eight entries: end-effector position, axis-angle orientation and the two gripper joint positions. Actions are the seven raw LIBERO OSC delta-action entries, with gripper -1 for open and $+1$ for close. The six-logit stage head uses six-way cross-entropy during training, masking only samples with no stage label. Deployment alone restricts the argmax to the task's K classes. The target head has no visibility or confidence output; missing training targets are handled by the target-loss mask.

Rendering. Marks are drawn in the stored 256×256 image before model preprocessing. A crosshair has a red center dot of radius 2 pixels and four green arms of length 10 pixels, with a 6-pixel gap and 2-pixel thickness. A box is a red 2-pixel outline placed outside the target box where possible; at image boundaries the band moves inside. Reversed box endpoints are sorted before rendering. Text and visual marks are independently retained with probability 0.5, so the four presence/absence combinations each have probability 0.25 before missing-target masking.

Recorded feedback procedure. Figure 1 specifies the inference rule used for every reported result. Its time index counts new action-chunk queries. The evaluator calls the policy at each environment step, but the auxiliary heads and stage filter update only when a new chunk is produced. Predictions from query j are rendered on the observation at query $j+1$, without tracking or geometric warping. The target is refreshed at every query independently of whether the filtered stage advances.

Recorded self-prompt inference. Input: policy π, table $\mathcal{T} = \{(s_k, m_k)\}_{k=0}^{K-1}$, execution horizon h.
Initialize filtered stage $k = 0$, candidate $c = \varnothing$, run length $r = 0$, target $z = \varnothing$.

1. At each new query, observe the main and wrist images and proprioception. Append the table text s_k to the instruction. Draw mark type m_k at the previous target z on a copy of the current main image; use a clean copy if $z = \varnothing$.
2. Predict the action chunk A with π. Read the final action-generation step's action-projection input features; the auxiliary heads produce logits $u \in \mathbb{R}^6$ and target $\hat{z} \in [0, 1]^6$.
3. Set $p = \arg\max_{0 \le a < K} u_a$ and set $z = \hat{z}$ if all its entries are finite, otherwise $z = \varnothing$.
4. If $p > k$, set $r \leftarrow r + 1$ when $p = c$, otherwise $r \leftarrow 1$; then set $c \leftarrow p$. If $r \ge 2$, set $(k, c, r) \leftarrow (p, \varnothing, 0)$. If $p \le k$, instead set $(c, r) \leftarrow (\varnothing, 0)$.
5. Execute the first h actions, stopping if the episode ends, then return to step 1.

Figure 1. The historical feedback rule. The initial query has stage-0 text and a clean prompt image. The filtered stage may skip intermediate stages and never decreases. This specifies the recorded implementation rather than a subsequently modified filter.

8. Task-table and label limitations

The task table provides ordered stage texts and mark types at test time. The stage classifier selects within that table; it does not infer an unseen task decomposition. RoboCasa labels preserve the demonstrated order, whereas the test table uses the majority order: reversed orders occur in 53/500 PreSoakPan, 156/500 LoadDishwasher, and 61/500 ArrangeTea demonstrations. A positional stage ID can therefore correspond to different semantic subtasks in training. This is an unresolved interface mismatch and prevents attributing floor-level results solely to a short training budget.

The original feedback implementation chooses mark type from the filtered stage but uses the latest target output, even when the raw stage prediction disagrees. Point and box coordinates are supervised on their respective mark types, making cross-type mismatch a potential source of incorrect marks. All tables report that original implementation. No result from a corrected feedback rule is included.

9. Run record

The host rebooted during the run, which wiped its RAM disk. Finished arms and final checkpoints had been backed up and were not rerun. Evo-1 LIBERO P was retrained from scratch on a dataset regenerated from the same raw demonstrations by the same procedure; state and action statistics match the original exactly. RoboCasa test scenes were regenerated from the same seeds, and all 80 match the original state hashes; RoboCasa pairing holds at the scene level, not the trajectory level. We removed 162 asset-error rows and 8 out-of-memory rows, all at zero steps, and reran those episodes. Read literally, our unknown-episode rule would void Evo-1 LIBERO N0; the estimate would then be $+2.0$ points and the criterion would still not be met. One Evo-1 LIBERO N0 first frame differs from the other arms; that state lies outside the common states, so it enters no contrast. Evo-1 evaluation ran partly on a GPU that was restored after the reboot and did not pass the standard render probe; 80 of its 81 first frames, the exception being the one above, and its scene states match the reference values.

10. Persistent History in the Trained X-VLA Policy

Purpose and attribution. This additional experiment asks whether the existing BranchBox-trained X-VLA policy can use persistent episode history. It transfers the main MemBodied architecture [2], with an explicit X-VLA backbone mapping. We retain all four components: associative reads, dedicated state/memory tokens, an episode-first anchor, and transition-conditioned writes. The architecture is not proposed as an independent invention. The same-weight history-reset intervention follows the diagnostic principle used by SmoLSTM [1].

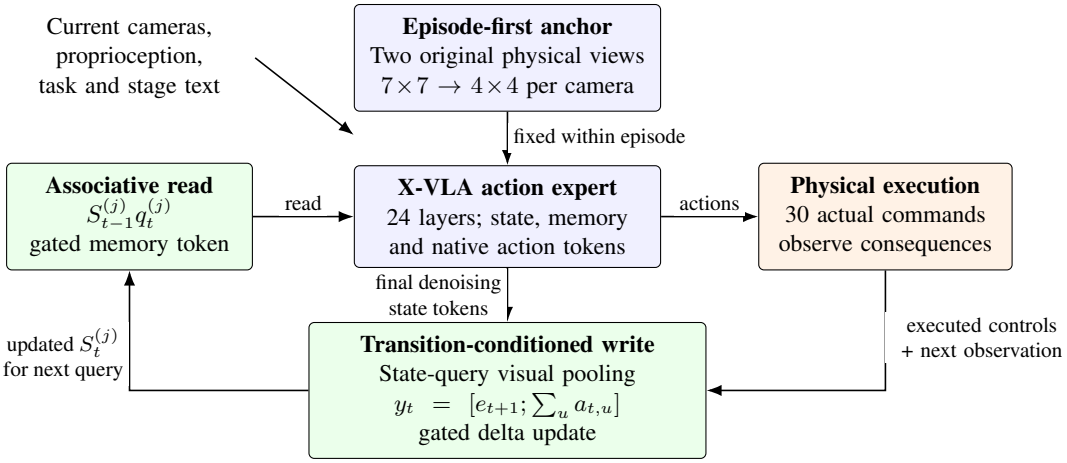


Figure 2. MemBodied adaptation in X-VLA-P. The recurrent matrix is read before action prediction and written only after physical execution and the next observation. Initial-scene features provide a separate fixed anchor. All denoising iterations share one matrix state; predicted but unexecuted controls never enter the write. Memory reset removes both persistent paths at every query.

Read and action prediction. Each of the 24 action-expert layers maintains a matrix $S_t^{(j)} \in \mathbb{R}^{128\times128}$. A projected proprioceptive token and a dedicated memory token are appended to the native action, visual, language and soft-prompt tokens. Existing native token positions are preserved. In the equations below, $f_x(z) = W_x z + b_x$ denotes a learned affine map. Let $h^{(j)}$ denote the normalized state token before the layer's self-attention. The read is

$$q^{(j)} = \mathrm{norm}_2\Big(\tanh(f_q^{(j)}(h^{(j)}))\Big), \tag{1}$$

$$r^{(j)} = 2\,\sigma(f_g^{(j)}(h^{(j)}))\, f_m^{(j)}(S_{t-1}^{(j)} q^{(j)}). \tag{2}$$

The scalar read gate modulates the projected memory at the dedicated memory-token position before attention. The factor two is the reference scale-to-rank ratio $256/128$. The learned initial matrices and memory tokens are initialized from $\mathcal{N}(0, 0.02^2)$; the readout and read-gate affine parameters are initialized to zero, initially giving a gate of 0.5 and zero readout. The action expert predicts the native 20-dimensional clean-action representation; only the seven physical action dimensions receive the masked action loss. At inference, ten denoising iterations share the same matrix state and anchor. Only the final iteration's post-layer state tokens are retained for a later write. Training uses one forward pass per query, with noise sampled across all 20 dimensions after padding the physical action.

State-query visual pooling. The ordinary X-VLA visual pass processes the main, prompt and wrist views. The memory path uses only the two physical camera views. After dropping the projector's global token, each physical camera contributes its actual 7×7 spatial patch grid. A query from the current proprioceptive state attends to learned key/value projections of these patches, with the two camera summaries then averaged. The patch tensors are detached only where they enter memory pooling; gradients through the ordinary visual-language action path remain enabled. Neither frozen neural features nor the rendered prompt image is substituted for an observed physical consequence.

Original first-frame anchor. The anchor is constructed from the original episode-first main-camera and wrist observations. Their native visual features are adaptively average-pooled from 7×7 to 4×4 patches per camera, giving 32 fixed anchor tokens. Anchor extraction uses the current visual encoder in evaluation mode with stop-gradient; the ordinary observation path still trains the encoder. Current state-query visual features query this anchor through trainable rank-64 cross-attention. The fused result is projected, repeated across action tokens, and concatenated with the native action/proprioception/time input. The widened affine layer copies the original coefficients and initializes the added anchor coefficients to zero. The anchor features remain fixed until episode reset. Random training windows use their actual episode-first images, not the first frame of the sampled window.

Execution-conditioned write. After a chunk has physically executed, the next observation supplies the new pooled visual feature e_{t+1}. Let $a_{t,u}$ be an actually executed command, transformed with the saved training normalizer $(a - \mu)/(\sigma + \epsilon)$. The write input is $y_t = [e_{t+1}; \sum_u a_{t,u}]$. At inference, $h^{(j)}_{t,\text{out}}$ is the preceding final-denoising state token; during training it comes from the preceding query's single noisy forward pass. We compute a unit-norm key, a value, and vector gates:

$$k_t^{(j)} = \text{norm}_2\Big(\tanh(f_k^{(j)}(h_{t,\text{out}}^{(j)}))\Big), \tag{3}$$

$$v_t^{(j)} = f_v^{(j)}(y_t), \tag{4}$$

$$\alpha_t^{(j)} = \sigma\Big(f_\alpha^{(j)}([h_{t,\text{out}}^{(j)}; y_t]) + 2\Big), \tag{5}$$

$$\beta_t^{(j)} = \sigma\Big(f_\beta^{(j)}([h_{t,\text{out}}^{(j)}; y_t]) - 1.5\Big). \tag{6}$$

With $\widetilde{S}_t^{(j)} = \text{Diag}(\alpha_t^{(j)}) S_{t-1}^{(j)}$, the update is

$$S_t^{(j)} = \widetilde{S}_t^{(j)} + \text{Diag}(\beta_t^{(j)})\big(v_t^{(j)} - \widetilde{S}_t^{(j)} k_t^{(j)}\big) k_t^{(j)\top}. \tag{7}$$

At inference, the state is initialized from learned matrices at every episode boundary and persists throughout that episode. Training instead resets it at the start of each sampled window. The harness records commands only after the environment step succeeds. One subsequent query authorizes one write; no predicted but unexecuted suffix and no success label enters memory. All 30 native X-VLA actions are executed between queries. A terminal partial chunk has no subsequent write.

Backbone adaptations. The published main architecture uses a different VLA and visual grid. Here the native Florence features are 7×7, so anchor pooling uses adaptive bins, which can overlap, instead of claiming literal 16×16 patch parity. Native X-VLA attention remains bidirectional; appending tokens preserves its existing positions but does not reproduce the reference block-causal attention mask. We retain X-VLA's 30-action execution cadence and one recurrent state per layer. The public reference's LIBERO configuration disables the anchor; this experiment explicitly keeps the complete main architecture. It therefore tests an X-VLA adaptation, not exact reproduction of the reference LIBERO recipe.

Full-weight sequence training. Training starts from the existing X-VLA-P checkpoint. All 941,771,580 parameters of the visual encoder, language encoder, action expert, original auxiliary heads, and new memory components are trainable. This comprises 30,172,376 memory parameters and 31,457,280 added affine coefficients beyond the 880,141,924-parameter starting policy. We use raw camera pixels with the original saved preprocessing. Training prompts replay the existing policy's earlier predictions on demonstrations, identically for every sampled sequence; current or future ground-truth annotations never construct input prompts. Current stage and target annotations supervise their respective auxiliary losses only. Demonstration actions supply the native noisy-action input and MSE target; past demonstrated controls may enter a write after their corresponding next observation is available.

The original seed-1 split assigns 160 complete demonstrations to this additional fine-tuning and 43 to monitoring. The initial checkpoint had previously trained on all 203; the 43 demonstrations are not unseen to the starting model. Sampling spans six consecutive queries at 30-action intervals, retaining the full recurrent gradient graph within each window. Tail padding is masked from action loss, auxiliary losses and writes. The effective batch is 32 sequences on four GPUs, eight sequences per rank. Global valid-action coordinates, labeled stages and valid targets have separate loss denominators, including across gradient accumulation and ranks. The objective remains native clean-action MSE plus 0.1 stage cross-entropy and 0.1 target L1.

AdamW uses $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 10^{-10}, gradient clipping at one, and EMA decay 0.99. The first 800 updates use 80 warmup updates, a peak learning rate of 2.5×10^{-5}, and cosine decay towards 2.5×10^{-6}. The next 1,200 updates start at the saved learning rate and decay to 2.5×10^{-7}. A subsequent compute-budget extension adds 4,000 updates: update 2,001 starts at the saved rate, updates 2,001–2,200 linearly increase it to 2.5×10^{-6}, and updates 2,201–6,000 use cosine decay back to 2.5×10^{-7}. Each continuation restores the raw model, AdamW moments and counters, EMA, and each rank's exact random-number state. The final extension explicitly revises the learning-rate schedule; it is not an unchanged 6,000-step schedule chosen before training. Both budget extensions were decided during training, before any new rollout evaluation. The master seed remains one; distributed dropout uses rank-derived streams initialized with $1 + \mathrm{rank}$, whose states are restored without reseeding. Only the final 6,000-update EMA is selected for evaluation; monitoring losses do not select the checkpoint.

Storage interruption and exact recovery. The original continuation reached update 1,200, but its save-space guard rejected a checkpoint write. The last complete checkpoint was update 1,000. Recovery restored that checkpoint's raw weights, optimizer, EMA and all four ranks' random-number states, retaining the original global learning-rate and sample sequence. Updates 1,001–1,200 were replayed; their recorded losses, gradient norms and input hashes match the discarded tail. Thus the completed trajectory retains 6,000 optimizer updates, while 6,200 updates were executed including the discarded 200-update tail. This infrastructure recovery did not depend on policy success. Later saves use immutable RAM snapshots with verified durable disk copies; the original checkpoints and failed-attempt logs are preserved.

Fixed development-panel comparison. The comparison uses the same four task IDs and official initial-state IDs 40–44: 20 paired scenes and 60 main rollouts. Each scene has three conditions: the existing X-VLA-P checkpoint, the full memory checkpoint, and the same full-memory weights with both added memory paths reset at every query. Memory reset reinitializes the matrix, discards pending hidden states/actions, and rebuilds the anchor from the current frame. Thus the intervention removes both recurrent and initial-scene history. It does not identify the two mechanisms separately. The first query of full memory and reset must match exactly. Because training uses persistent history within each window, repeated inference-time reset also changes the distribution of memory states and anchors. A positive full-versus-reset difference would demonstrate dependence of this trained policy on those histories, rather than establish an advantage over an independently trained memory-free policy.

Each rollout uses a fresh environment, literal seed one, ten settling steps, the same initial-state vector and initial pixels, and a 520-step horizon. All three conditions for a scene execute on the same GPU; scene assignment alternates between two GPUs and condition order follows a fixed six-permutation rotation. The original BranchBox stage filter and one-query target feedback remain enabled in all three conditions. Their algorithms and lifecycles are shared; subsequent stage and target predictions can differ. Reset removes the added MemBodied history, rather than making the entire controller history-free. Raw executed controls, per-query memory and image hashes, prompt transitions, and initial lossless frames are retained. Repeated engineering replays are excluded from the 60 main outcomes. The original policy receives no additional fine-tuning, so its contrast with the final model measures the combined effect of additional training and the memory-equipped architecture. The same-weight intervention measures the trained policy's use of the added historical paths.

For each of the two fixed contrasts, pointwise 95% percentile intervals resample the five paired scenes within each task, with 100,000 replicates and bootstrap seed one. We also report discordant-pair counts and two-sided exact McNemar tests. Holm adjustment applies to the two p-values, not to the intervals. The 20 scenes appeared in earlier pilots; they constitute a repeated development panel, not an unseen confirmatory evaluation. Intervals condition on this single training run and these tasks. Complete paired outcomes, including regressions, accompany the analysis.

The retained demonstration artifacts do not include their complete simulator initial states. Exact overlap with the 20 official evaluation initial states therefore could not be established from those artifacts; we make no claim that training and evaluation initial states are disjoint. Episode identifiers and robot proprioception alone are insufficient for this check.

10.1. Complete memory-panel results

On the separate 20-scene development panel, the final memory model solves 12/20 episodes, compared with 13/20 for the existing X-VLA-P and 11/20 with the same weights and both added memory paths reset. Relative to the existing checkpoint, the paired difference is -5.0 percentage points (pointwise 95% CI $[-30.0, +20.0]$; 4 rescued scenes and 5 regressions; Holm-adjusted exact McNemar $p = 1.000$). Relative to the same-weight reset, the paired difference is $+5.0$ percentage points (pointwise 95% CI $[-15.0, +25.0]$; 3 rescued scenes and 2 regressions; Holm-adjusted exact McNemar $p = 1.000$). The existing-checkpoint comparison includes additional full-weight training. The same-weight intervention measures the combined use of recurrent and initial-scene history; it does not isolate the anchor or compare against an equally fine-tuned memory-free model. These are conditional development-panel estimates from one training run. All paired outcomes and task-level results appear in the supplementary material.

Task	Existing	Full memory	History reset	Memory–existing (pp)	Memory–reset (pp)
Moka pots	4/5	5/5	4/5	$+20.0$	$+20.0$
Two mugs	0/5	2/5	0/5	$+40.0$	$+40.0$
Mug and pudding	5/5	2/5	3/5	-60.0	-20.0
Soup and sauce	4/5	3/5	4/5	-20.0	-20.0

Table 9. All four tasks in the fixed development panel. Each cell has five scenes; pp denotes percentage points.

Task	Initial-state ID	Existing	Full memory	History reset
Moka pots	40	0	1	1
Moka pots	41	1	1	1
Moka pots	42	1	1	1
Moka pots	43	1	1	0
Moka pots	44	1	1	1
Two mugs	40	0	1	0
Two mugs	41	0	0	0
Two mugs	42	0	0	0
Two mugs	43	0	1	0
Two mugs	44	0	0	0
Mug and pudding	40	1	1	1
Mug and pudding	41	1	0	1
Mug and pudding	42	1	0	0
Mug and pudding	43	1	0	0
Mug and pudding	44	1	1	1
Soup and sauce	40	1	1	1
Soup and sauce	41	1	0	1
Soup and sauce	42	1	0	0
Soup and sauce	43	1	1	1
Soup and sauce	44	0	1	1

Table 10. All 60 main outcomes, with 1 denoting success and 0 failure. Engineering replays are excluded. Rows are paired by task and initial-state ID.

References

[1] Jan-Gerrit Habekost, Parsa Mastouri Kashani, Connor Gäde, Matthias Kerzel, Philipp Allgeuer, Cornelius Weber, Stefan Wermter, and Jae Hee Lee. SmoLSTM: A compact vision-language-action model with recurrent memory that persists, 2026. arXiv:2609.22854. 6

[2] Tej Deep Pala, Navonil Majumder, Bryce Goh, Raphael Yee, Jianfei Yang, Liming Chen, and Soujanya Poria. MemBodied: Recurrent associative memory for vision-language-action models, 2026. arXiv:2609.28256. 6